

StudyBuddy Chatbot

CM2015 Programming with Data - Midterm Coursework Report

Chatbot title/domain: StudyBuddy is a university study planning and programming coursework support chatbot. It helps students plan coursework, understand Python/chatbot implementation concepts, prepare revision tasks, manage deadline pressure, and inspect short-term memory stored during the conversation.

Deliverables: The project package contains a Jupyter notebook, an external `intents.json` dataset, this PDF report, and an exported HTML copy of the notebook.

1. Application and use case

StudyBuddy is designed for students who need quick guidance while building or revising programming coursework. The target user may ask for help with planning a project, understanding regex or JSON, writing tests, coping with workload stress, or summarising stored session details. The chatbot is intentionally rule-based so that its behaviour is transparent and easy to test.

The chatbot does not attempt to replace a tutor or provide official university policy. Its role is to give structured, lightweight prompts that help the user decide the next action, such as writing one JSON intent, running the notebook, or adding a test case.

2. Code and data organisation

The notebook is organised into clearly labelled sections: imports, JSON loading, preprocessing, pattern matching, memory and NLP helpers, response generation, main loop, demonstration transcript, automated tests, and reflection. The external dataset stores all intent tags, regex patterns, and response templates.

Component	Purpose	Evidence in project
<code>intents.json</code>	Stores tags, regex patterns, and response templates.	16 intents including <code>greeting</code> , <code>deadline_help</code> , <code>python_help</code> , <code>testing_help</code> , <code>memory_summary</code> , <code>keyword_request</code> , and <code>fallback</code> .
<code>load_intents()</code>	Loads and validates the JSON dataset.	Checks required keys and list types before the chatbot runs.
<code>build_chatbot_resources()</code>	Builds compact dictionaries for matching and responses.	Creates <code>intent2responses</code> , <code>intent2description</code> , and <code>pattern2intent</code> dictionaries.
<code>find_intent()</code>	Searches compiled regex patterns.	Uses <code>re.IGNORECASE</code> and returns the match object for named capture groups.
<code>generate_response()</code>	Selects and formats a response.	Uses <code>random.choice</code> and string template substitution from <code>memory/NLP</code> context.
<code>run_chatbot()</code>	Interactive loop.	Keeps asking for input until the user types <code>exit</code> or <code>quit</code> .

3. Pattern recognition and preprocessing

Pattern recognition is handled with regular expressions stored in the JSON file and compiled at runtime. The patterns are flexible enough to recognise variations in wording and punctuation. For example, the `deadline` intent includes a named group for numeric deadlines, using a construct equivalent to `\d{1, 3}`, and another pattern uses lookaheads so that both a `project` word and a `deadline` word can appear in any order.

The preprocessing pipeline uses regex tokenisation to split user input into word-like units, removes punctuation, removes common stop words, and stems tokens using NLTK PorterStemmer when available. These processed tokens are used for keyword extraction, sentiment scoring, and fallback behaviour.

This design separates exact intent recognition from broader text analysis. Regex handles deterministic intent detection, while preprocessing supplies lightweight NLP features that make the chatbot more informative.

4. Response generation and memory

Responses are selected randomly from a list of templates for the detected intent. This prevents the chatbot from returning the same sentence every time. Several templates contain placeholders such as {name}, {course}, {deadline}, {keywords}, and {sentiment_label}.

Short-term memory is stored in a Python dictionary and updated using named regex groups. For example, the input "My name is Maya" captures the name and stores it as `memory["name"]`. A later memory query can then produce a response such as: "Current memory: name=Maya, course=CM2015, favourite colour=not set, last deadline=in 10 days."

5. Advanced features

- Named regex groups are used for memory extraction, including name, course, colour, topic, and numeric deadline context.
- A small lexicon-based sentiment analyser labels the tone of a message as positive, neutral, or negative. Negative fallback messages can be routed to the workload-support intent.
- Keyword extraction uses tokenisation, stop-word removal, stemming, and frequency counts to summarise the most important words from the user input.
- A safe formatting dictionary prevents the program from crashing if a response template contains an unexpected placeholder.
- Automated tests check behaviour rather than exact random wording, which makes the test suite robust despite random response selection.

6. Test cases

The notebook includes an executable test suite. The table below summarises three representative tests that illustrate chatbot behaviour. Additional tests in the notebook cover keyword extraction and fallback handling.

Test case	User input	Expected behaviour	Why it matters
Memory update	My name is Maya	Intent is <code>set_name</code> ; memory stores <code>name=Maya</code> ; response has no unresolved template placeholders.	Demonstrates named regex capture, dictionary memory, and dynamic string substitution.
Deadline planning	Can you plan my CM2015 project deadline in 10 days?	Intent is <code>deadline_help</code> ; deadline memory becomes "in 10 days"; response gives project-submission planning advice.	Demonstrates flexible regex, numeric capture using digits, and project-relevant response generation.
Stress support	I feel anxious and overwhelmed about the workload	Intent is <code>motivation_stress</code> ; sentiment label is negative; response suggests a smaller next action.	Demonstrates basic sentiment analysis and an advanced response feature beyond simple pattern matching.

7. Process reflection

The project was developed through a week-by-week iterative process.

Week 1 - domain and first prototype

The first decision was to create a study-support chatbot because it naturally fits a student-facing programming assignment. The first prototype used a small number of hard-coded responses. Self-review feedback showed that this would not satisfy the file-usage and maintainability expectations, so the intent data was moved to an external JSON file.

Week 2 - modular functions and regex

The second iteration introduced reusable functions for loading intents, compiling patterns, searching patterns, generating responses, and running the main loop. Testing showed that exact matching was too brittle. The regex patterns were therefore expanded with optional words, grouping, digit matching, case-insensitive compilation, and a lookahead-based deadline pattern.

Week 3 - preprocessing, memory, and NLP

The third iteration added tokenisation, stop-word removal, stemming, short-term memory, random response selection, sentiment scoring, and keyword extraction. Feedback from manual testing showed that responses felt repetitive and impersonal, so random templates and memory substitutions were added.

Week 4 - tests, report, and export

The final iteration focused on evidence. The notebook now contains a reproducible demo transcript and automated tests. Tests assert intent tags, captured memory, sentiment labels, and absence of unfilled placeholders rather than one exact sentence, because response text is chosen randomly. The notebook was exported to HTML and packaged with the JSON dataset and report.

8. Running and reproducibility

The project uses relative paths. The notebook expects `intents.json` to be in the same folder as the notebook. The implementation imports standard Python libraries plus NLTK for Porter stemming if available. No NLTK data download is required, so the notebook can run in a normal Jupyter environment without internet access.

To run the project, open `StudyBuddy_CM2015_midterm.ipynb`, run all cells from top to bottom, and then optionally execute `run_chatbot()` in a new cell. The interactive loop prints an opening message, accepts user

input, and exits only when the user types `exit` or `quit`.

9. Conclusion

StudyBuddy satisfies the main technical goals of the coursework: it uses dictionaries and lists to manage chatbot data, modular functions to keep code organised, regex for pattern recognition, external JSON for file usage, preprocessing and stemming for basic NLP, dynamic response generation with memory, and tests to verify behaviour. The extra sentiment and keyword features provide additional intelligence while keeping the program understandable and reproducible.